

Министерство науки и высшего образования Российской Федерации

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №8

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Обход графа в ширину»

Выполнили:

студенты групп 24BBB3

Масюк А. Р.

Мамонтов Н. П.

Приняли:

Юрова О. В.

Деев М. В.

Пенза 2025

Цель работы

Освоить на практике методы обхода графов в ширину с использованием матриц смежности и списков смежности на языке C++. Закрепить навыки работы с очередью из стандартной библиотеки для реализации алгоритма BFS и анализа порядка посещения вершин.

Лабораторное задание

Задание:

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру обхода в ширину, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс queue из стандартной библиотеки C++.
3. * Реализуйте процедуру обхода в ширину для графа, представленного списками смежности.

Работа программы

```
C:\Users\admin\source\repos\118\118\x64\Del
input number of verts: 10
0 1 1 0 0 1 0 0 0 0
1 0 0 1 1 1 1 1 1 1
1 0 0 0 1 0 1 0 0 1
0 1 0 0 0 0 1 0 0 1
0 1 1 0 0 1 0 1 0 1
1 1 0 0 1 0 0 1 1 1
0 1 1 1 0 0 0 0 1 1
0 1 0 0 1 1 0 0 0 1
0 1 0 0 0 1 1 0 0 1
0 1 1 1 1 1 1 1 1 0
```

Матрица смежности графа

```
0 1 1 1 1 1 1 1 1 0
Input start vert: 2
Path: 2 0 4 6 9 1 5 7 3 8
```

Поиск от начальной вершины

Вывод

Освоили на практике методы обхода графов в ширину с использованием матриц смежности и списков смежности на языке C++. Закрепили навыки работы с очередью из стандартной библиотеки для реализации алгоритма BFS. Реализовали генерацию случайных графов, их преобразование между различными представлениями и визуализацию порядка обхода вершин. Написали рабочий код, точно следующий теоретическому описанию алгоритма обхода в ширину.

Листинг

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <conio.h>
#include <locale.h>
#include <queue>

using namespace std;

void BFS(int** G, int numG, int* visited, int s) {
    queue<int> q;
    int v;

    visited[s] = 1;

    q.push(s);

    while (!q.empty()) {
        v = q.front();
        q.pop();
        printf("%3d", v);

        for (int i = 0; i < numG; i++) {
            if (G[v][i] == 1 && visited[i] == 0) {
                q.push(i);
                visited[i] = 1;
            }
        }
    }
}

int main() {
    setlocale(LC_ALL, "Russian");

    int** G;
    int* visited;
    int numG, current;

    printf("input number of verts: ");
    scanf("%d", &numG);

    visited = (int*)malloc(numG * sizeof(int));
    G = (int**)malloc(numG * sizeof(int*));

    for (int i = 0; i < numG; i++) {
        G[i] = (int*)malloc(numG * sizeof(int));
    }

    for (int i = 0; i < numG; i++) {
        visited[i] = 0;

        for (int j = i; j < numG; j++) {
            G[i][j] = G[j][i] = (i == j ? 0 : rand() % 2);
        }
    }

    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
```

```
        printf("%3d", G[i][j]);  
    }  
  
    printf("\n");  
}  
  
printf("Input start vert: ");  
scanf("%d", &current);  
  
printf("\nPath: ");  
  
BFS(G, numG, visited, current);  
printf("\n\n");  
  
for (int i = 0; i < numG; i++) {  
    free(G[i]);  
}  
  
free(G);  
free(visited);  
  
_getch();  
return 0;  
}
```